



CV Assistant AI

Asistente inteligente de revisión y mejora de CVs

Reto 9 — IA Generativa

Artificial Intelligence Foundations · Fundació URV · Mayo 2026

¿Por qué un revisor de CVs con IA?

El problema real

El candidato

- Invierte horas elaborando su CV
- No tiene acceso fácil a feedback profesional
- Un CV mal estructurado → descartado en segundos

El coste del feedback experto

- Coach de carrera: **60–150 € / sesión**
- No siempre accesible para todos los perfiles

El lado del recruiter

*Los recruiters dedican una media de **6–10 segundos** a la revisión inicial de un CV*

- Priorizan estructura clara y logros cuantificables
- CVs sin palabras clave del sector → descartados automáticamente

La oportunidad

Acceder al feedback mediante IA Generativa: análisis profundo, en segundos, de forma gratuita.



Demo en vivo

Flujo completo de la aplicación

Demo — Flujo a mostrar

Paso a paso

1. Sidebar de configuración

- Sector: **Tecnología** · Puesto: **Senior Developer** · Experiencia: **5 años**

2. Subir CV → *test_cv_example.txt*

3. Pulsar "Analizar CV" → spinner de progreso

4. Recorrer resultados:

-  Puntuación general
-  Análisis por secciones
-  Fortalezas /  Áreas de mejora
-  Sugerencias concretas
-  Sección mejorada — original vs reescrita

5. Descargar análisis en formato JSON



Stack tecnológico y decisiones de diseño

Arquitectura de la Solución

Stack tecnológico

Capa	Tecnología	Rol
Frontend	 Streamlit	UI, upload, visualización, estado de sesión
LLM	 Gemini 3.1 Flash Lite (OpenRouter)	Análisis del CV
Lenguaje	 Python 3.11	Backend completo
Extracción	PyPDF2 + python-docx	Lectura PDF y DOCX
Prompting	<code>prompt_templates.py</code>	7 técnicas de prompt engineering
Parseo	 Parseo del json	JSON + 3 niveles de fallback
Despliegue	 Streamlit Community Cloud	Hosting gratuito desde GitHub

Módulos del Proyecto

Arquitectura modular en 5 componentes

Módulo	Responsabilidad única
 app.py	Orquestador: UI, gestión de estado, pipeline
 llm_integration.py	conexión OpenRouter API, parámetros
 prompt_templates.py	Prompt y analizador
 parsers.py	Extracción JSON, validación y fallback de 3 niveles
 utils.py	Extracción texto PDF / DOCX / TXT, limpieza

Decisiones Técnicas Clave

¿Por qué estas tecnologías y no otras?

Decisión	Elegido 	Descartado 	Razón
Framework UI	 Streamlit	FastAPI+React, Gradio	Python puro, componentes nativos, despliegue integrado
Modelo LLM	 Gemini 3.1 Flash Lite	 Llama 3.1 70B (Groq)	Groq deprecated el modelo durante el desarrollo
API de LLM	 OpenRouter	OpenAI direct	Gratuito (500 req/día), compatible con SDK OpenAI
Despliegue	 Streamlit Cloud	Heroku, AWS	Gratuito, nativo Streamlit, secrets integrados
Parseo	 3 niveles fallback	json.loads()simple	El LLM se desvía ocasionalmente del formato

Decisión más impactante: `temperature = 0.2`
Reducir de 0.7 → 0.2 eliminó prácticamente todos los fallos de parseo JSON.
Coherencia y determinismo por encima de creatividad en tareas estructuradas.

Prompt Engineering

7 técnicas aplicadas y su impacto medido

Prompt Engineering (1/2)

Técnicas y su impacto

Técnica	Problema que resuelve	Impacto
Role Prompting	Vocabulario y criterios genéricos	Calibra tono, nivel de exigencia y vocabulario de RRHH
Context Setting	Evaluación descontextualizada	Relativiza la puntuación según sector, puesto y experiencia
Chain-of-Thought	Output redundante y desorganizado	Coherencia entre los 8 campos del JSON
Output Schema JSON	Campos renombrados, decimales, estructura errónea	Fuerza contrato de datos estricto

SYSTEM: "Eres un experto senior en recursos humanos y revisión de CVs con más de 15 años de experiencia. Siempre proporcionas respuestas ÚNICAMENTE en formato JSON válido, sin texto adicional."

Prompt Engineering (2/2)

Técnicas y su impacto (continuación)

Técnica	Problema que resuelve	Impacto
Few-Shot Examples ×3	Modelo excesivamente benevolente (7–8/10 para CVs mediocres)	Calibra escala 0–10: bajo / medio / alto + 3 sectores distintos
Instrucciones Negativas	JSON envuelto en ```json, decimales, datos inventados	Reduce fallos de parseo de ~30% a <5%
Separación de datos	Prompt injection desde el CV	CV al final, delimitado con <code>---</code> , post-instrucciones
Temperature = 0.2	Variabilidad en formato JSON	Maximiza determinismo; elimina fallos de parseo residuales

CRÍTICO: Devuelve ÚNICAMENTE el objeto JSON.
Cero texto antes ni después. No uses bloques ```json```.
Si el JSON no puede parsearse directamente, tu respuesta es inválida.

Sistema de Parseo — 3 Niveles de Fallback

Requisito del reto: NO mostrar texto crudo al usuario

```
Respuesta del LLM
↓
Nivel 1: json.loads() directo      → ✓ ~90% de respuestas bien formadas
↓ fallo
Nivel 2: regex extrae bloque ```json``` → ✓ ~8% con markdown residual
↓ fallo
Nivel 3: regex extrae { ... }      → ✓ ~1.5% con texto prefijo/sufijo
↓ fallo
Fallback: get_fallback_analysis()  → análisis básico de error (~0.5%)
```

Mapeo JSON → componentes Streamlit

Campo del LLM	Componente Streamlit
puntuacion_general	st.metric
analisis_secciones	st.expander × 3 (experiencia, educación, habilidades)
fortalezas_principales	st.success (lista)
areas_mejora`	st.warning (lista)
seccion_mejorada	st.text_area comparativa (original vs mejorada)

Resultados y Validación

Ejemplo de análisis real

CV analizado: Data Scientist · 3 años de experiencia · sector Tecnología

Campo	Valor
Puntuación general	6 / 10
Puntuación Experiencia	7 / 10
Puntuación Educación	8 / 10
Puntuación Habilidades	4 / 10
Nº sugerencias generadas	6

Observación clave

Sin few-shot examples, el modelo puntuaba este mismo CV con **8/10** y generaba sugerencias genéricas del tipo *"mejora tu LinkedIn"*.

Con los 3 examples calibrados, la puntuación bajó a **6/10** y las sugerencias fueron 3× más concretas y accionables.

Los examples calibran la escala. Sin ellos, el modelo es siempre benevolente.



Conclusiones

Lo que aprendimos

Conclusiones

✓ Aprendizajes clave

El prompt engineering importa más que el modelo

La calidad del output depende más del diseño del prompt que del tamaño del modelo.

Temperature baja = JSON consistente

Reducir de 0.7 → 0.2 eliminó prácticamente todos los fallos de parseo.

Few-shot examples calibran la escala

Sin ejemplos, el modelo es sistemáticamente benevolente.

El parseo robusto es obligatorio en producción

El LLM se desvía del formato ocasionalmente; el fallback es imprescindible.

🔧 Trabajo futuro

- **Comparación CV vs ofertas reales**
Integrar LinkedIn/Indeed API para contextualizar el análisis con requisitos de mercado
- **Soporte multilingüe**
Análisis en inglés, francés y otros idiomas
- **OCR para CVs escaneados**
Procesamiento de PDFs basados en imagen
- **Historial persistente entre sesiones**
Base de datos para seguimiento de mejoras a lo largo del tiempo

¡Gracias!

 [Links del proyecto](#)

 [App en producción](#)

 [Repositorio GitHub](#)

Javier Martín Castro · Artificial Intelligence Foundations · Fundació Universitat Rovira i Virgili · Mayo 2026